AI-SOC: Major Project Plan (Rechecked)

Shashwat Shukla · B.Tech CSE, University of Mumbai · Plan date: 28 Oct 2026 · Source: AI-SOC README (all 883 lines read)

1. What the project is

AI-SOC takes Windows failed-logon events (Event ID 4625) through a Python monitor into a Spring Boot API, stores them in PostgreSQL, applies a threshold rule to raise incidents, and shows them in a React dashboard together with an investigation result. Today that investigation is a mock. Ollama with qwen2.5:3b has been tested on its own but is not connected to the backend.

What makes it a major project rather than a dashboard: the LLM is only an advisor. Evidence, deterministic rules, a policy engine, human approval and an audit trail surround it, and only allow-listed, bounded actions can ever run. Your own principle: AI proposes, evidence supports, rules verify, safety controls constrain, humans approve high-risk actions.

Suggested thesis statement: Evidence-grounded, policy-constrained LLM investigation and bounded remediation for Windows authentication attacks, evaluated with a small local model.

The measurable claim to prove: a small local model (3B) can be used safely in SOC triage when wrapped in schema validation, evidence grounding and deterministic policy, and the wrapping measurably reduces unsupported or unsafe output. Your own Qwen test already showed the problem (wrong ATT&CK ID, an unsupported 'no known malicious activity' claim, inconsistent severity and risk). Measuring and fixing that is your central result.

Assumptions in this plan (change if wrong): Stage I review around mid-December 2026 and final submission/viva around late April 2027; work is mostly solo; development on one Windows machine with Ollama, plus room for one or two VMs. Shift dates to your college calendar.

2. Where the project stands

Done and verified (per README):

- Python monitor reads Event ID 4625 and POSTs alerts to /api/alerts

- Alerts stored in PostgreSQL; rule creates an incident at 5 FAILED_LOGON alerts within 5 minutes
- Incident list API and React dashboard
- Mock investigation saved to PostgreSQL, retrievable at /api/incidents/{id}/investigation, survives backend restart
- Ollama 0.40.8 and qwen2.5:3b installed and run independently

Not done:

- Ollama connected to the backend; strict schema and output validation
- Correlation by source IP, account and host; duplicate-incident prevention
- Policy engine, approval workflow, real remediation, audit trail (README marks these planned or not verified end to end)
- Authentication, migrations, CI, deployment

3. Recheck: problems found in the README and design

These are the gaps I found when reading the whole file against itself. Each one is fixed somewhere in the plan below.

1. **Detection is the weakest foundation.** Global counting means unrelated events merge and duplicates appear. Everything downstream (LLM evidence, which IP to block, which account to disable) depends on correct grouping, so this is fixed first.
2. **The mock contradicts your own principles.** The sample response shows incident severity HIGH from detection but investigation severity MEDIUM, and recommends BLOCK_SOURCE_IP with approvalRequired false, although section 17 says disruptive actions need approval. Its evidence lines (same external IP, multiple accounts) are hardcoded, while the real rule does not correlate by IP or account. The mock must be built from real incident data.
3. **Too many decisions sit inside the investigation result.** Severity, technique, risk and approvalRequired are all produced by the investigator, and the Qwen test showed a 3B model gets them wrong. Move them to deterministic code: the detection rule fixes the ATT&CK technique and a severity floor, and the policy engine computes approval.
4. **Repo structure and status disagree.** ResponsePlanService, PolicyService and ApprovalService are listed in the structure, but status says policy and approval are planned or unverified, and section 12 says other endpoints may exist. A Week 1 audit must mark each as real, partial or stub and publish the true route table.

5. **Everything lives in one package** (ai_soc_backend/incident). Split by area (ingestion, detection, incident, investigation, policy, approval, remediation, audit, security) so the modularity claim is true.
6. **No authentication is described.** If anyone can POST /api/alerts, anyone can forge an alert and steer remediation at an arbitrary IP. Monitor-to-backend needs an API key and the dashboard needs login and roles.
7. **Prompt injection is not mentioned.** Usernames, workstation names and other log fields are attacker-controlled and will be placed in the prompt. Treat them as data, constrain output by schema, and let the policy engine ignore anything the model says that is not an allowed action.
8. **Monitor reliability.** No bookmark of the last event read, no retry queue, no idempotency key. A restart can duplicate or lose events. The backend should reject a repeated (host, event record id).
9. **A local test cannot show 'block source IP'.** Failed logons on the dev machine often carry no useful remote address (4625 can show - or ::1 for local logons; LogonType 2, 3 and 10 behave differently). You need a second VM generating failures over the network. Verify this in week 1 because it is on the critical path.
10. **Overclaiming in the title.** 'Autonomous Remediation' is in the headline but remediation is unimplemented. The plan delivers lab-only bounded remediation; keep the disclaimer.
11. **Test results are 'previously passed'.** Add CI so they are rerun on every push.
12. **Setup hygiene.** Hardcoded D:\ paths, a placeholder clone URL, ddl-auto schema updates, runs only from IntelliJ. Use relative paths, .env.example, Flyway migrations, and Docker Compose.

4. Scope and cut line

Must (the project is not credible without these):

- Detection v2 with correlation and no duplicate incidents; reliable monitor
- Ollama-backed investigation with schema, grounding and consistency validation, plus safe fallback
- Deterministic policy engine (default deny)
- Login, roles, approval workflow and UI
- Append-only audit log

- Lab remediation: dry-run plus one real action on an isolated VM, with verification, timeout and rollback
- Evaluation with real numbers
- Report, presentation, demo video

Should:

- Evidence enrichment, incident timeline, MITRE ATT&CK lookup from a local file (RAG-lite)
- Two more rules (password spraying, success after failures)
- Model comparison (3B versus a 7B or 8B model if hardware allows)
- Analytics dashboard, analyst feedback buttons

Could (only if ahead): vector-DB RAG, Linux or application logs, SIEM integration, cloud LLM second opinion.

Won't: production hardening, real production remediation, automatic retraining.

Cut rule: if you are more than one week behind at any gate, drop items from the bottom of Should first. Never cut Must.

5. Target design and key decisions

D1. Who decides what

- Detection rules decide that an incident exists, its baseline severity and its ATT&CK technique
- The LLM supplies a classification from a fixed list, a summary, findings that cite evidence IDs, clearly marked hypotheses, a proposed action from a fixed list, and a confidence
- The policy engine decides final severity, whether the action is allowed, and whether approval is needed
- Humans approve or reject; the executor runs only allow-listed actions

D2. Investigation pipeline

1. Build an evidence pack from the incident's alerts and compute the facts in code (counts, distinct IPs and accounts, time span, logon types, any success after failures), so the model never has to count.
2. Fill a versioned prompt template; wrap evidence in delimiters and state that it is data, not instructions.

3. Call Ollama with temperature 0, a fixed seed, a JSON schema in the format field, and a timeout.
4. Validate: schema, enums, ranges.
5. Check grounding: every cited evidence ID exists; any IP, account or number in the text appears in the evidence pack; reject claims about threat intelligence that was never provided.
6. Check consistency: severity versus risk matrix; technique equals the rule's technique.
7. On failure: one retry with the validation error, then a rule-based fallback result labelled FALLBACK and NEEDS_REVIEW.
8. Store the raw response, prompt version, model, latency and validation report; keep history instead of overwriting.

D3. Policy engine

- Table-driven (YAML or DB), versioned, default deny, no dependency on the LLM so its tests run without Ollama
- Input: incident facts, proposed action, target facts, confidence, environment mode
- Output: ALLOW, REQUIRE_APPROVAL or DENY with reason codes
- Example rules: BLOCK_SOURCE_IP is never automatic; private, loopback, own-network or allow-listed IPs are DENY; disabling a protected account is DENY; confidence below a threshold means analyst review; a global kill switch denies all remediation

D4. Response state machine PROPOSED, then POLICY_CHECKED, then either DENIED or PENDING_APPROVAL, then APPROVED or REJECTED or EXPIRED, then EXECUTING, then VERIFIED or FAILED, then ROLLED_BACK where applicable. Illegal transitions are rejected in code.

D5. Remediation executor

- Separate module with its own allow-list of action types and strict parameter validation
- Actions: DRY_RUN, NOTIFY, BLOCK_SOURCE_IP (Windows Firewall rule on the lab VM named by action ID, auto-removed after a TTL such as 30 minutes), DISABLE_TEST_ACCOUNT (lab only)
- Idempotency key per incident, action and target; rate limit per hour; verification step that queries whether the rule or account state really changed; rollback; every step audited
- Mode flag REMEDIATION_MODE defaults to DRY_RUN

D6. Audit log Append-only table with actor (system, model, user), event type, payload and the hash of the previous row so tampering is detectable. Block UPDATE and DELETE at the database level.

D7. Platform security Spring Security with roles ANALYST (view, feedback), APPROVER (approve, reject), ADMIN (policy and config). Monitor uses an API key. CORS locked down, input validation, rate limits, secrets only from environment variables.

6. Data model and API additions

Tables: alerts (host, event_record_id, event_time, source_ip, target_user, logon_type, status, raw_json; unique on host plus record id), incidents (rule, correlation key, severity, status, first_seen, last_seen, alert_count), incident_alerts, investigations (versioned, with source LLM or FALLBACK or MOCK, model, prompt_version, raw_response, validation_report), response_plans, policy_decisions, approvals, remediation_actions, audit_events, users and roles.

Endpoints to add: investigate (POST), response-plan (GET and POST), policy evaluation (internal), approvals (list, approve, reject), remediation actions (status and rollback), audit (read-only, filtered), metrics, auth (login), analyst feedback.

7. Work plan with dates and exit gates

Phase 0: Baseline and hygiene (2 to 8 Nov)

- Run the full pipeline once end to end and record screenshots and logs; tag v0.1-baseline
- Audit the repo: mark every service and controller as real, partial or stub; correct README status and add the true route table
- Rerun backend tests and the frontend build; add GitHub Actions for both
- Environment-based secrets, .env.example, relative paths, Flyway baseline replacing ddl-auto=update; Postgres in Docker
- Critical-path check: create one VM and confirm a network logon failure gives a 4625 with a real source IP
- Create a task board from this plan
- **Exit:** CI green, README matches code, source IP confirmed

Phase 1: Detection and ingestion v2 (9 to 22 Nov)

- Normalized alert schema; monitor keeps the last record ID, retries with backoff, sends an API key, handles missing IPs
- Idempotent ingest on (host, event_record_id); validate payloads and handle missing timestamps explicitly
- Sliding-window correlation per key; incident lifecycle: one open incident per rule and key, new alerts attach to it, auto-close after a quiet period
- Rules: R1 brute force against one account from one source; R2 password spraying (one source, many accounts); R3 success after failures (needs 4624 collection). Thresholds in config
- Table-driven tests: exactly at threshold, window edges, two interleaved attackers, duplicate delivery, out-of-order timestamps
- **Exit:** replaying 200 synthetic alerts gives the expected incident count with zero duplicates and separate incidents per attacker

Phase 2: Ollama integration and validation (23 Nov to 13 Dec)

- OllamaInvestigationService behind the existing interface; config switch mock, ollama or ollama-with-fallback; mock rebuilt from real incident data
- Evidence pack builder, prompt v1 in a versioned file, JSON schema, timeouts
- Validators from D2; a small ATT&CK allow-list file (T1110, T1110.001, T1110.003, T1078 to start)
- Persist raw output and validation report; dashboard shows a source badge, validation warnings, and observed evidence separately from hypotheses
- Tests with a stubbed Ollama (WireMock): valid, malformed JSON, wrong enum, invented IP, timeout, connection refused; each must end in a safe result and no action
- Evaluation harness plus 30 labelled scenarios
- **Exit:** every failure mode yields a fallback, never an exception or an action; first accuracy table

Stage I milestone (14 to 20 Dec): working pipeline with real LLM investigation and validation; report chapters 1 to 5; presentation; short demo.

Buffer (21 Dec to 3 Jan): exams and holidays. Only tests, docs and report writing.

Phase 3: Policy, response plan, audit (4 to 17 Jan)

- Policy tables and engine, reason codes, versioning; response plan generated from a validated investigation

- State machine with enforced transitions; hash-chained audit log written by every component
- Decision-table unit tests; a test proving the engine gives the same answer with no LLM running
- **Exit:** a proposed action to block a private or allow-listed IP is always DENY; every decision appears in the audit log

Phase 4: Authentication, roles, approvals (18 to 31 Jan)

- Spring Security, roles, API key for the monitor, CORS and validation
- Approval inbox, approve and reject with a reason, expiry of stale requests; audit view and incident timeline in the UI
- Authorization test matrix: every role against every endpoint
- **Exit:** an unapproved action can never reach the executor; forged alert without a key is rejected

Phase 5: Lab remediation (1 to 21 Feb)

- Week 1: isolated host-only lab (target VM, attacker VM), VM snapshots, executor skeleton with DRY_RUN and allow-list
- Week 2: real firewall block on the target VM with TTL, verification and rollback; DISABLE_TEST_ACCOUNT
- Week 3: abuse and failure tests (duplicate actions, rate limit, kill switch, executor offline, verification failure)
- **Exit:** scripted attack leads to an approved block that is verified, then removed on expiry, all audited

Phase 6: Evidence enrichment and hardening (22 Feb to 7 Mar)

- Evidence provenance and timestamps; timeline view; ATT&CK lookup from a local file
- Prompt-injection test set (hostile usernames, instructions inside fields) and fixes
- **Exit:** injection scenarios cause no policy bypass and no schema escape

Phase 7: Evaluation v2 and packaging (8 to 21 Mar)

- Expand to 80 to 100 scenarios; run all configurations; model comparison; red-team results; latency
- Analytics dashboard (only if on schedule); full Docker Compose for Postgres, backend, frontend and optionally Ollama (the Windows monitor stays native)
- **FEATURE FREEZE on 21 Mar**

Phase 8: Documentation and viva (22 Mar to 25 Apr)

- Final report, architecture and sequence diagrams, presentation, 5 to 7 minute demo video, viva question sheet, repo cleanup, tag v1.0
- Two weeks of buffer for fixes and rehearsals

8. Testing and evaluation plan

Test layers: unit tests (rules, validators, policy), integration tests with Testcontainers PostgreSQL, stubbed-Ollama contract tests, end-to-end browser tests, authorization matrix, chaos tests (kill Ollama, kill the DB, duplicate deliveries), CI on every push.

Scenario dataset (80 to 100 JSON event sets with ground truth): single-account brute force, password spraying, brute force followed by success, benign typos, lockout after a password change, misconfigured service account looping, low-and-slow attacker, two attackers at once; plus adversarial cases: hostile text in a username, empty evidence, conflicting evidence. Optionally add public Windows event samples (EVTX-ATTACK-SAMPLES, OTRF Security-Datasets) for realism after checking their licences.

Configurations to compare (ablation):

- A: rule-only or mock
- B: raw LLM output, no validation
- C: LLM plus schema validation
- D: full system (schema, grounding, deterministic fields, policy)
- E: D with a larger model, if hardware allows

Metrics: schema-valid rate; classification accuracy and macro-F1; technique accuracy; unsupported-claim rate; severity and risk consistency; fallback rate; p50 and p95 latency; incident precision, recall and duplicate rate; policy-correct decisions; **unsafe actions executed (must be 0)**; remediation success, verification and rollback rates; MTTD, MTTI and MTTR from recorded timestamps. Run each LLM configuration three times to report variance. Report failures honestly; the 3B model's mistakes are a finding, not a flaw in the report.

9. Lab and safety rules

- Remediation only on dedicated lab VMs in a host-only network; snapshot before each test

- Generate failed logons only against your own lab machines and test accounts; never against external systems or real accounts
- Default mode DRY_RUN; protected list for gateway, host machine and admin accounts; every block has a TTL; global kill switch
- No production or personal data in logs or datasets; no secrets in Git
- Keep the README disclaimer about educational use and no real blocking until controls are tested

10. Risks and mitigations

- **Scope too large:** cut line in section 4 and phase exit gates
- **Small model quality:** deterministic fields, validation, fallback, optional larger model; it becomes a result
- **Slow inference on your hardware:** run investigations asynchronously with a status field, strict timeouts, cache results
- **Cannot generate a real remote source IP:** lab VM verified in week 1
- **Remediation locks you out:** VMs only, TTL, protected list
- **Exams and holidays:** buffer weeks already scheduled
- **Lost work:** push daily, tag every milestone, dump the database before each phase
- **Demo failure:** seeded replay script and a recorded video as backup

11. Deliverables and submission checklist

- GitHub repository with tags v0.1 (baseline), v0.5 (Stage I), v1.0 (final) and a rewritten README
- Diagrams: system context, containers, and a sequence from alert to verified remediation
- Report: introduction, literature survey (SOC automation and SOAR, LLMs in security operations, MITRE ATT&CK, prompt injection, human-in-the-loop control), requirements, design, implementation, testing and evaluation, results, limitations and ethics, conclusion and future work
- Presentation, demo video, evaluation data and scripts, viva question sheet
- Any plagiarism or AI-use declaration your college requires

Likely viva questions to prepare: why a local model; why the LLM does not decide; how prompt injection is handled; what happens when Ollama is down; how accuracy was measured; how this differs from existing tools such as Wazuh active response or commercial SOAR; what stops a forged alert from causing a block.

12. Demo script (final)

1. Attacker VM produces failed logons against the target VM
2. One incident appears, not several
3. Investigation shows an LLM badge, cited evidence, and hypotheses kept separate
4. Proposed action BLOCK_SOURCE_IP; policy returns REQUIRE_APPROVAL
5. Approver approves in the UI
6. Executor adds the firewall rule; verification confirms it; attacker is blocked
7. Audit trail shows every step; the rule expires and is rolled back
8. Negative cases: malformed model output leads to fallback; proposal to block the gateway is DENIED; hostile username is ignored; unauthenticated alert is rejected

13. Next 7 days

1. Rerun the full pipeline and record the baseline; tag v0.1-baseline
2. Audit services and controllers; update the README status table and route list
3. Add CI for backend tests and frontend build
4. Move secrets to environment variables; add .env.example; replace D:\ paths in the README
5. Create the lab VM and confirm a 4625 with a real source IP
6. Write the normalized alert schema and the unique constraint on (host, event_record_id)
7. Create the task board with every item above

14. Two-person split (replaces the solo assumption in section 1)

Dates, phases and exit gates stay exactly as in section 7. The two people are called Person A and Person B below; swap the labels to match your strengths, but keep the split by area so each person owns features from database to screen.

Principle: split by ownership boundary, not by phase. Person A owns everything that detects and acts (the safety-critical backend). Person B owns everything that thinks and shows (the LLM, evaluation, login, approvals and UI). Each person reviews the other's pull requests, so both can explain the whole system in the viva.

Person A: Platform, Policy and Response

Owns: Windows monitor, ingestion, detection, incident, policy, remediation and audit packages; lab VMs; CI and database migrations.

- Phase 0: CI pipeline, Flyway baseline, environment-based secrets, lab VM that produces a 4625 with a real source IP
- Phase 1: all of it (monitor bookmark, retry and API key; idempotent ingest; correlation; rules R1, R2, R3; incident lifecycle; edge-case tests)
- Phase 2: starts the policy engine early using the agreed investigation schema; also Docker Compose for Postgres and backend
- Phase 3: finishes policy engine, response plan, state machine, hash-chained audit log
- Phase 5: remediation executor, firewall block with TTL, verification, rollback, kill switch, abuse and failure tests
- Phase 7: evaluation scenarios and metrics for detection, policy and remediation (duplicate rate, unsafe actions executed, MTTD, MTTR)

Person B: Intelligence, Security and Experience

Owns: investigation, approval and security packages; the whole frontend; evaluation harness.

- Phase 0: audit of frontend and investigation code; frontend build in CI
- Phase 1: builds incident fixtures so work does not wait for Detection v2; scenario dataset v1; dashboard cleanup (filters, incident detail); prototypes the prompt and schema directly against Ollama
- Phase 2: all of it (OllamaInvestigationService, evidence pack, prompt v1, JSON schema, validators, fallback, stubbed-Ollama tests, source badge and warnings in the UI)
- Phase 4: Spring Security and roles, API key validation for the monitor endpoint, approval workflow, approval inbox, audit view, authorization test matrix
- Phase 6: evidence provenance, incident timeline, ATT&CK lookup, prompt-injection test set and fixes

- Phase 7: expand dataset to 80 to 100 scenarios, ablation runs, model comparison, analytics dashboard, full Docker Compose

Joint work

- **Week 1 (by 8 Nov), pair on contracts before anyone builds:** write the shared schemas and an OpenAPI file so both sides can work in parallel
- **Phase 0 audit:** A audits backend services and controllers, B audits frontend and investigation code, then merge into one corrected README status table and route list
- **Phase 8:** report assembly, demo rehearsals, viva preparation

Interfaces to agree in week 1

1. **Alert schema:** defined by A, consumed by B (fixtures and evidence pack)
2. **Incident evidence object** (incident plus its alerts and computed facts): provided by A, consumed by B
3. **Investigation result JSON schema:** defined by B, consumed by A's policy engine
4. **Policy decision and response plan format:** defined by A, shown in B's UI
5. **Action state machine and approval API:** A owns the state enum and executor; B owns the approval endpoints and UI
6. **Audit event format:** defined by A; B writes audit events from investigation, approval and login

After 8 Nov, changing any of these needs both people's agreement in a pull request.

Unblocking rules

- B builds against fixture incidents until Detection v2 lands; A uses the mock investigation, rebuilt from real data, until the Ollama service lands
- A tests the executor with a test-only approved action until the approval workflow lands
- Anything not ready by its gate is stubbed behind the agreed interface, never worked around

Working agreement

- Short-lived feature branches; every pull request reviewed by the other person; CI must be green; CODEOWNERS file by package

- Flyway migrations named with date-based versions to avoid number clashes; shared hotspot files (application.properties, App.tsx, the AIInvestigation entity) change only in small, quick pull requests
- Two short syncs per week plus a one-hour integration session every Friday where main is run end to end and tasks can be moved between people
- Keep a contribution log of who built what, for any individual contribution statement your college asks for
- After each phase, each person teaches the other their part for 15 minutes

Report and viva split

- **Person A writes:** ingestion and detection design, policy and remediation design, lab and safety, results for detection and remediation
- **Person B writes:** literature survey on LLMs in security operations, investigation design, authentication and approvals, evaluation method and results for investigation
- **Both write:** introduction, architecture, limitations, conclusion; each reads the full report before submission
- **Demo:** A runs the attack and remediation half, B runs the investigation and approval half; both prepare every viva question in section 11

What the second person changes

- Keep the dates; use the extra capacity to promote Should items to Must, in this order: ATT&CK lookup, model comparison, rules R2 and R3, analytics. Re-check the cut line at the Stage I milestone
- Load balance: B is lighter in Phase 1 and A is lighter in Phase 2, which is why each starts the other's upcoming work early (fixtures and dataset for B, policy engine for A). If either person falls more than one week behind, move tasks at the Friday session instead of waiting